



Facultad de
**Ciencias Sociales y
Tecnologías de la Información**
Talavera de la Reina. UCLM

PROYECTO ISO II

Pablo Gómez García / Álvaro Olmedo Fernández

Pablo Nieto Mayoral / Pablo Sánchez García-Ochoa

ÍNDICE

CONTENIDO

Metodología de desarrollo	3
Product Owner	3
Scrum Master	3
Equipo de Desarrollo	3
Reunión de Planificación del Sprint	4
Gestión de la Configuración del Software	4
Project Board - Scrum	5
Sprints	5
Sprint 0	5
Sprint 1	6
Sprint 2	6
Sprint 3	6
Sprint 4	6
Sprint 5	7
Requisitos.....	7
Funcionales.....	7
No Funcionales.....	8
DIAGRAMAS UML	9
DIAGRAMA de BD.....	10
Diagramas de Comunicación	10
Buscar alojamiento.....	10
Reservar alojamiento.....	10
Dar de alta propiedad.....	11
Diagrama de secuencia	11
Prototipo de sitio Web.....	12
Página principal	12
Inicio de sesión y Crear cuenta	13
ESTRUCTURA DE LA APLICACIÓN	17
Análisis del software	18
SonarCoud.....	18
Vincular GITHUB con SonarQube.....	18
Quality Profile	18
Quality Gate	26

TESTING	28
Gestor Propiedad	29
Gestor Reserva	29
Gestor Lista Deseos	29
Gestor Búsqueda	29
Gestor ROL	29
Gestor Notificación	29
Gestor Disponibilidad	29
MANTENIMIENTO	30

METODOLOGÍA DE DESARROLLO

En nuestro proyecto utilizaremos Scrum como metodología de gestión de proyectos junto con Kanban como metodología de desarrollo de software.

PRODUCT OWNER

Álvaro Olmedo será el mayor representante del producto.

Define las características del producto y establece los requisitos de este. Prioriza las características del proyecto respecto a su valor de mercado, a su vez establece la prioridad de los requisitos en cada iteración.

SCRUM MASTER

En nuestro caso es Pablo Gómez. Es el responsable de mantener el repositorio GitHub y se asegura de mantener activa la comunicación entre roles. A su vez, Pablo G. se asegura que el grupo permanece funcional.

EQUIPO DE DESARROLLO

Pablo Nieto y Pablo Sánchez se encargarán de desarrollar cualquier tipo de funcionalidad.

Serán ayudados en las tareas por Álvaro O. y Pablo G. en estas tareas.

REUNIÓN DE PLANIFICACIÓN DEL SPRINT

Las reuniones se harán diariamente variando su duración en cuanto a la disponibilidad de los integrantes, con el objetivo de realizar un sprint de 2 semanas.

GESTIÓN DE LA CONFIGURACIÓN DEL SOFTWARE

Con el uso de GitHub utilizaremos el desarrollo por ramas. Emplearemos 4 ramas principales:

- **Rama Developer:**

Esta rama principal permite la revisión en desarrollo o el mantenimiento del código. Permite también corregir los problemas detectados antes del lanzamiento del producto.

- **Ramas features:**

Estas ramas creadas desde developer sirven para implementar nuevas características o funcionalidades.

Cuando se ha completado el desarrollo de la nueva característica se une con la rama developer para liberarla en la siguiente versión.

- **Rama de release:**

Una vez se va a lanzar una nueva versión de la aplicación, se añade a esta rama para hacer pruebas y detectar posibles errores para poder corregirlos sin que afecte al usuario.

- **Rama main:**

Esta rama es la versión final en la que se lanza tras pasar por developer y release, contiene los cambios realizados pulidos de developer. Se genera una versión mediante un etiquetado y se lanza la aplicación.

PROJECT BOARD - SCRUM

Hemos usado un Project Board con estas columnas principales:

- ➔ **Backlog:** inventario maestro con todas las tareas.
- ➔ **Sprint / To Do:** ítems comprometidos para el sprint actual.
- ➔ **In Progress:** tareas que alguien está desarrollando.
- ➔ **QA / In Testing:** pruebas manuales / verificación.
- ➔ **Done:** tareas completadas según la Definition of Done.

SPRINTS

El proyecto se desarrollará a lo largo de 6 sprints de dos semanas cada uno (**PD:** en el sprint 3 lo empezamos dos semanas después de realizar la Entrega 1):

- ➔ **Sprint 0:** Sprint inicial para preparar la tecnología y los medios para poder trabajar de forma conjunta.
- ➔ **Sprint 1:** Definir cómo será la aplicación y dejar la estructura del proyecto lista para empezar a programar.
- ➔ **Sprint 2:** Implementar la funcionalidad principal mínima y los flujos básicos de usuario (registro/login o menú principal) para tener algo que usar y probar. (Entrega 1)
- ➔ **Sprint 3:** Continuar con la implementación de las funcionalidades restantes después de la entrega 1.
- ➔ **Sprint 4:** Verificar que todo lo desarrollado (excepto funcionalidad favoritos) funciona.
- ➔ **Sprint 5:** Terminar de probar las funciones que nos quedaban pendientes del anterior sprint, configurar Sonar, hacer testing, corregir los últimos fallos y preparar la entrega haciendo merge a la rama main y terminar la documentación.

SPRINT 0

Sprint inicial para preparar la tecnología y los medios para poder trabajar de forma conjunta.

- ➔ Planificación del sprint 0
- ➔ Revisión de sprint 0
- ➔ Crear diagrama de la Base de Datos
- ➔ Crear diagramas UML
- ➔ Configurar Spring JPA
- ➔ Configuración de Maven + Spring y Spring JPA
- ➔ Crear repositorio
- ➔ Crear estructura básica del repositorio
- ➔ Recogida de requisitos del proyecto
- ➔ Análisis de requisitos
- ➔ Planificar los Sprints

SPRINT 1

En este sprint hemos definido cómo será la aplicación y dejar la estructura del proyecto lista para empezar a programar.

- ➔ Planificación del sprint 1
- ➔ Revisión de sprint 1
- ➔ Crear la estructura mediante módulos, carpetas y base de datos
- ➔ Crear estructura inicial del proyecto
- ➔ Prototipo del diseño de la interfaz
- ➔ Diseño de la arquitectura básica

SPRINT 2

En este sprint hemos implementado la funcionalidad principal mínima y los flujos básicos de usuario (registro/login o menú principal) para tener algo que usar y probar.

- ➔ Planificación del sprint 2
- ➔ Revisión de sprint 2
- ➔ Implementación del backend del login
- ➔ Creación de la base de datos
- ➔ Creación de la página Login
- ➔ Creación de la página de Home

SPRINT 3

En este sprint continuamos con la implementación de las funcionalidades restantes después de la entrega 1.

- ➔ Planificación del sprint 3
- ➔ Revisión de sprint 3
- ➔ Implementación dar de alta una propiedad
- ➔ FrontEnd login propietario
- ➔ FrontEnd dar de alta una propiedad
- ➔ Implementación dar de alta propietario
- ➔ Implementación pago
- ➔ FrontEnd Pago
- ➔ FrontEnd reserva propiedad
- ➔ Implementación reserva propiedad

SPRINT 4

En este sprint hemos verificado que todo lo desarrollado (excepto funcionalidad favoritos) funciona.

- ➔ Planificación del sprint 4
- ➔ Revisión de sprint 4
- ➔ Probar funciones del sistema (excepto favoritos)
- ➔ Mejorar la documentación del proyecto
- ➔ Guardado y búsqueda de datos de la propiedad
- ➔ Guardar datos mediante la interfaz para probar las distintas funcionalidades.

SPRINT 5

En este sprint teníamos que terminar de probar las funciones que nos quedaban pendientes del anterior sprint, hacer testing, corregir los últimos fallos y preparar la entrega haciendo merge a la rama main y terminar la documentación.

- ➔ Planificación del sprint 5
- ➔ Revisión de sprint 5
- ➔ Corregir los últimos fallos antes de la entrega
- ➔ Entregar el proyecto final y retrospectiva
- ➔ Creación HTML vista Propietario
- ➔ Revisión entre los integrantes del grupo
- ➔ Probar funciones del sistema (excepto favoritos)
- ➔ Comprobar Errores comunes
- ➔ Implementación de visualizado de propiedades
- ➔ Testing GestorPropiedades
- ➔ Testing GestorRol
- ➔ Testing GestorPropietario
- ➔ Testing GestorReserva
- ➔ Documentación Testing
- ➔ Instalación de entorno de pruebas integrándolo con el proyecto Maven
- ➔ Configuración Sonar y Documentación
- ➔ Testing GestorBusqueda
- ➔ Testing GestorUsuario
- ➔ Testing GestorDisponibilidad
- ➔ Testing GestorListaDeseos
- ➔ Mantenimiento Preventivo

REQUISITOS

FUNCIONALES

RF01 - Los propietarios, después de dar de alta una cuenta personal, pueden asociar propiedades a la misma.

RF02 - Los propietarios, de manera excepcional, pueden aceptar a los usuarios para su servicio.

RF03 - Los usuarios pueden buscar alojamiento filtrando por: destino, fechas, tipo de inmueble... para examinar las opciones de alquiler.

RF04 - Además, los usuarios, de forma excepcional, pueden aplicar filtros avanzados. Ejemplo: ver aquellos inmuebles con posibilidad de reserva inmediata; seleccionar filtros para ciertas comodidades; o seleccionar política de cancelación de reserva.

RF05 - Los inmuebles poseerán una descripción en la que se detallan las características de esta y sus servicios.

RF06 - No es necesario tener una cuenta de cliente para poder ver los alojamientos, pero solo pueden almacenar propiedades en una lista de deseos personal.

RF07 - Es necesario tener una cuenta de usuario para poder aplicar a la reserva del inmueble deseado, no es posible hacerlo sin estar registrado en la plataforma.

RF08 - Los inquilinos una vez logeados, al seleccionar el inmueble a alquilar, pueden aplicar a la reserva.

RF09 - Las propiedades con reserva inmediata se puede completar el pago con tarjeta de crédito, débito o PayPal desde la plataforma.

RF10 - La aplicación no deberá permitir más de una única reserva por inmueble en el mismo periodo de tiempo.

RF11 - La aplicación no deberá permitir más de una reserva para una misma persona en el mismo periodo de tiempo.

RF12 - Los usuarios están asociados a la persona titular de la cuenta, la cual debe ser participe del uso del inmueble.

RF13 - Los propietarios podrán elegir las condiciones para la cancelación de las reservas, las cuales serán comunicadas al inquilino antes del pago. Se pedirá una confirmación de lectura al mismo para saber que es consciente de las mismas.

RF14 - Al terminar una estancia, se podrá puntuar a los usuarios por el estado de los inmuebles y a los inquilinos por el servicio prestado. A su vez, pueden añadir un breve texto argumentando el porqué de su valoración, la cual puede ser de ayuda a otros usuarios de la plataforma.

RF15 - La interfaz poseerá un menú desplegable para que los usuarios puedan acceder a las principales funcionalidades de la plataforma.

- Futuras especificaciones: Permitir el cambio de unidad monetaria a elección del usuario.

NO FUNCIONALES

RNF01 - El sistema debe permitir un uso fluido para los usuarios sin incluir tiempos de carga excesivos.

RNF02 - El sistema debe soportar grandes cantidades de usuarios en temporadas de alta demanda.

RNF03 - La interfaz debe ser intuitiva, sencilla, accesible desde el navegador y fácil de entender para los usuarios.

RNF04 - Planificar escalonadamente los cambios a realizar para no modificar código sin tener claro la idea que se busca.

RNF05 - Presentar código ordenado y documentado para ser entendible de modificar en actualizaciones futuras, fallos detectados o implementar nuevas funcionalidades.

RNF06 - Consistencia en los procesos a tiempo real, si un usuario reserva en una fecha el cambio debe reflejarse inmediatamente.

RNF07 - Generar diferentes ramas del proyecto para generar estabilidad entre versiones.

RNF08 - Antes de la publicación o actualización del código probar previamente las funcionalidades de este y buscar posibles errores del código modificado.

RNF09 - Escalabilidad que permita un mejor soporte para los usuarios en actualizaciones futuras.

DIAGRAMAS UML

Diagrama de Flujo de la página web de RentPlace

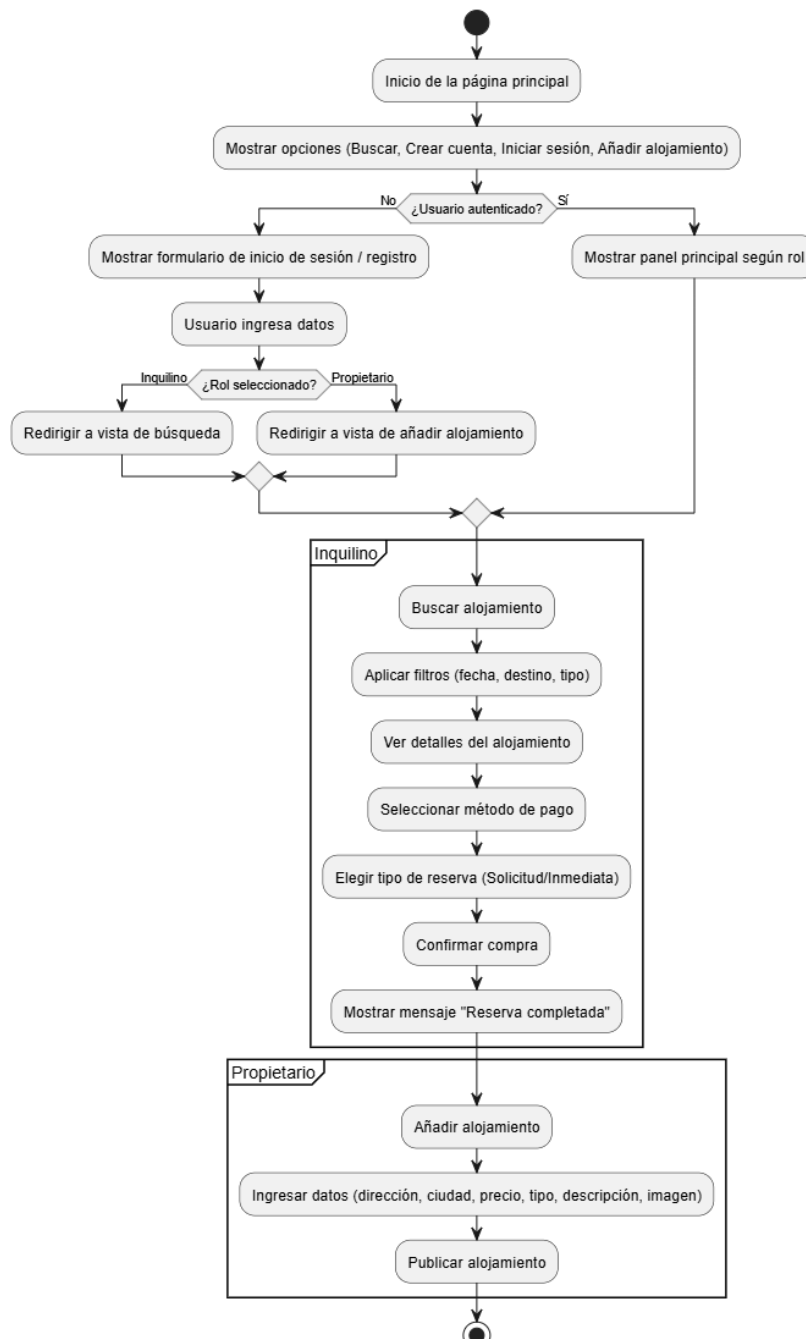
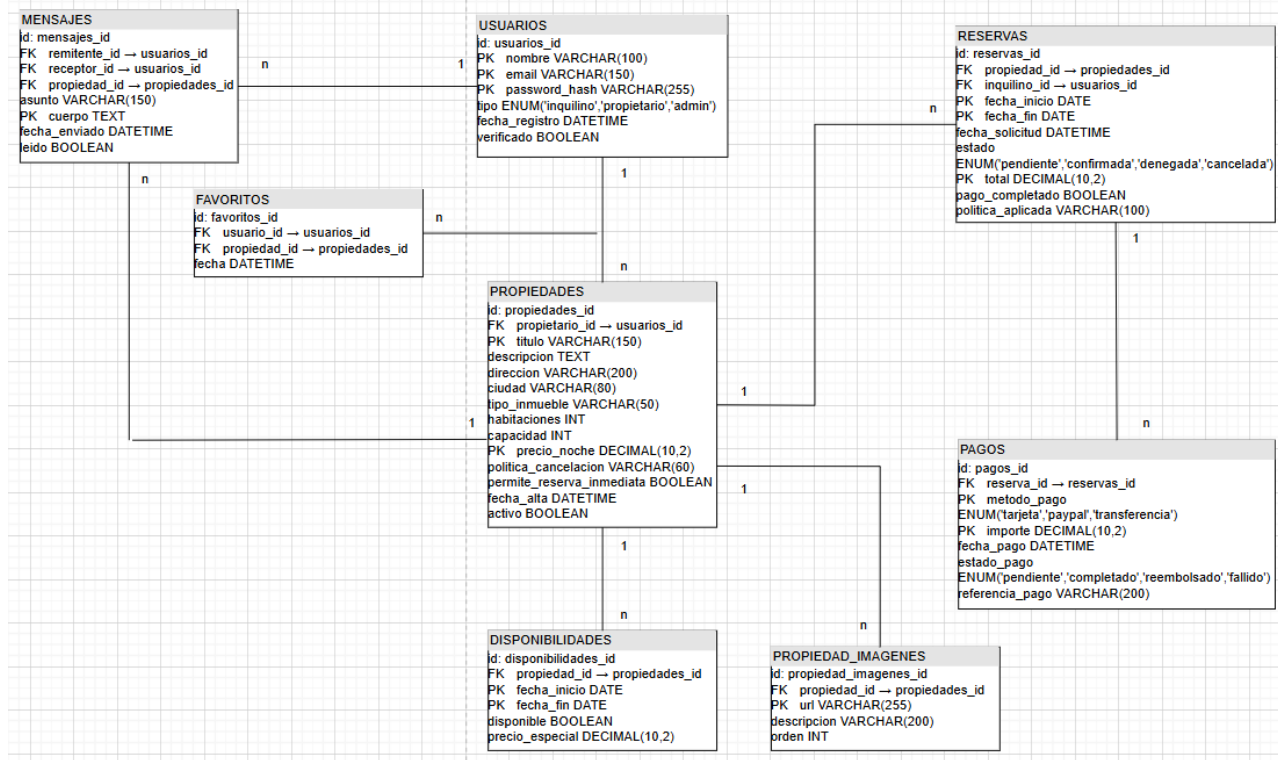


DIAGRAMA DE BD

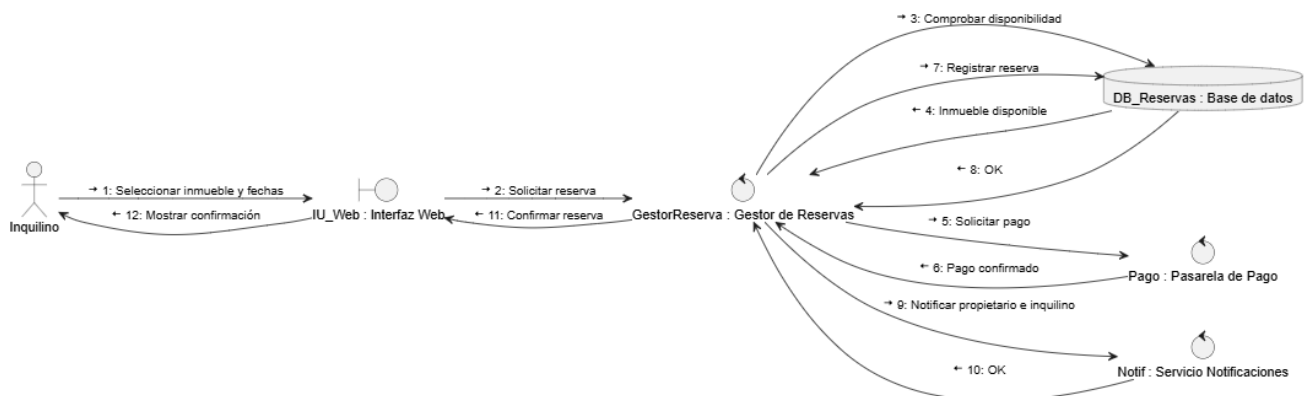


DIAGRAMAS DE COMUNICACIÓN

BUSCAR ALOJAMIENTO



RESERVAR ALOJAMIENTO



DAR DE ALTA PROPIEDAD

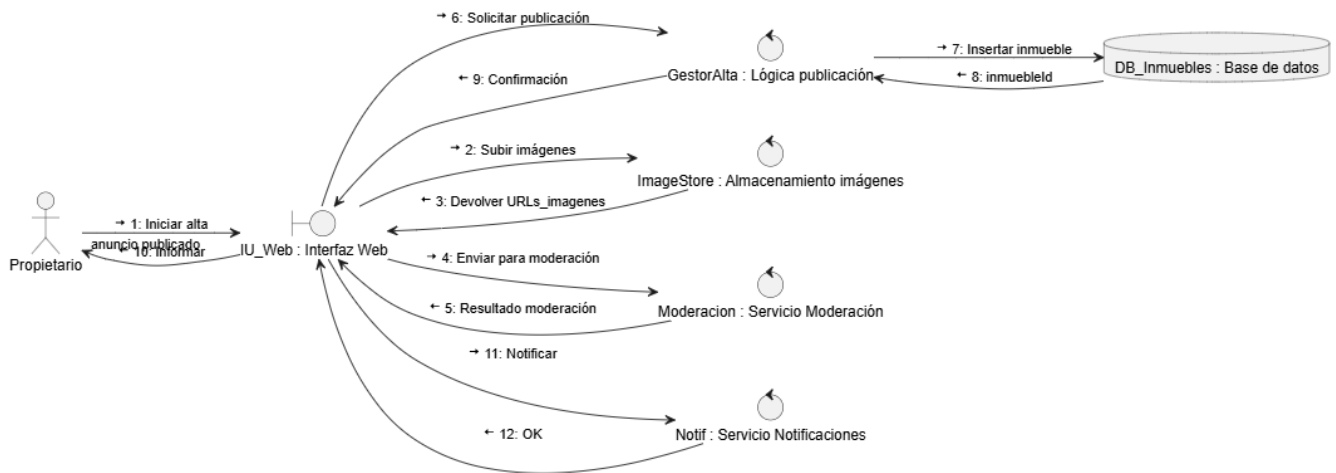
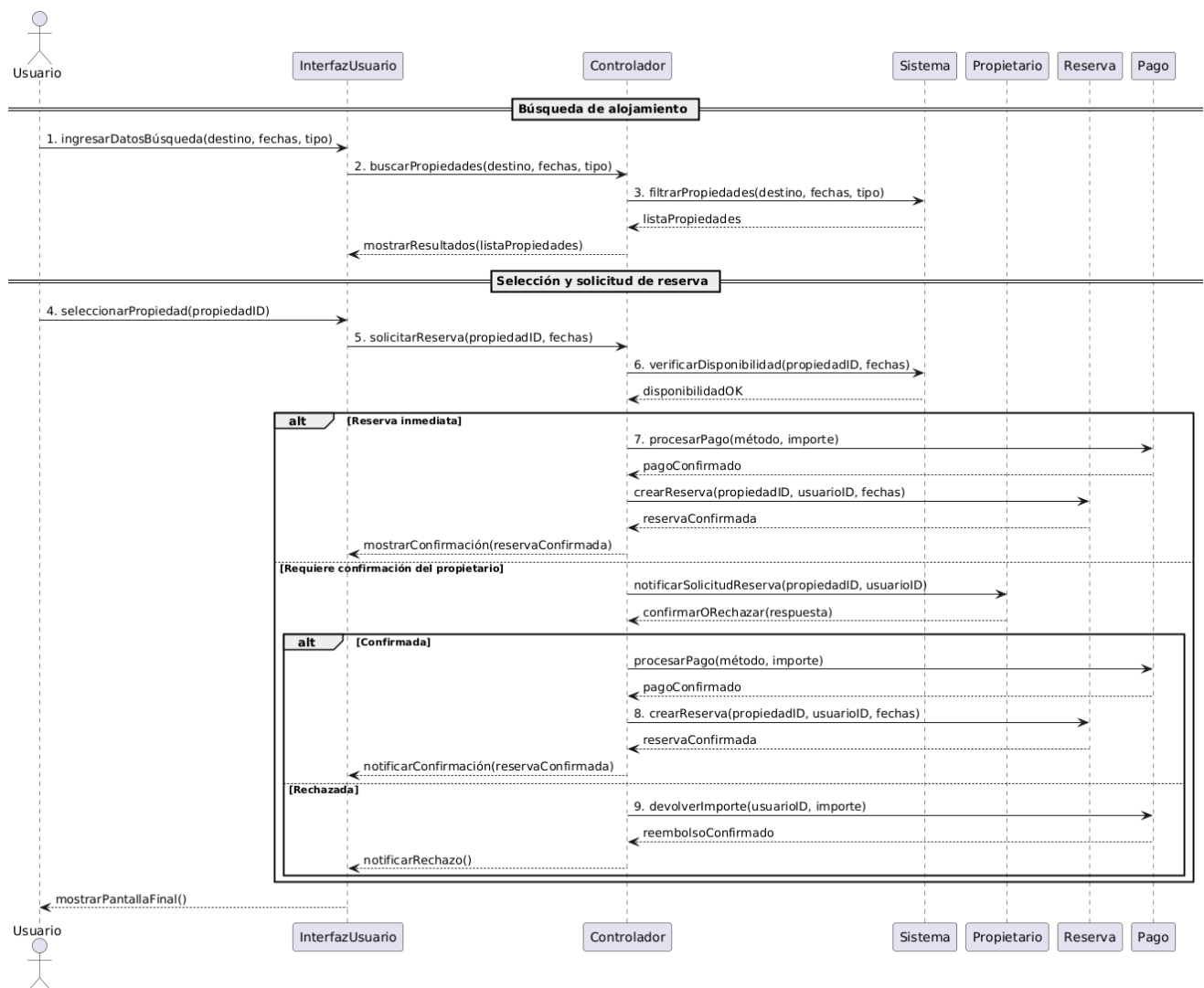
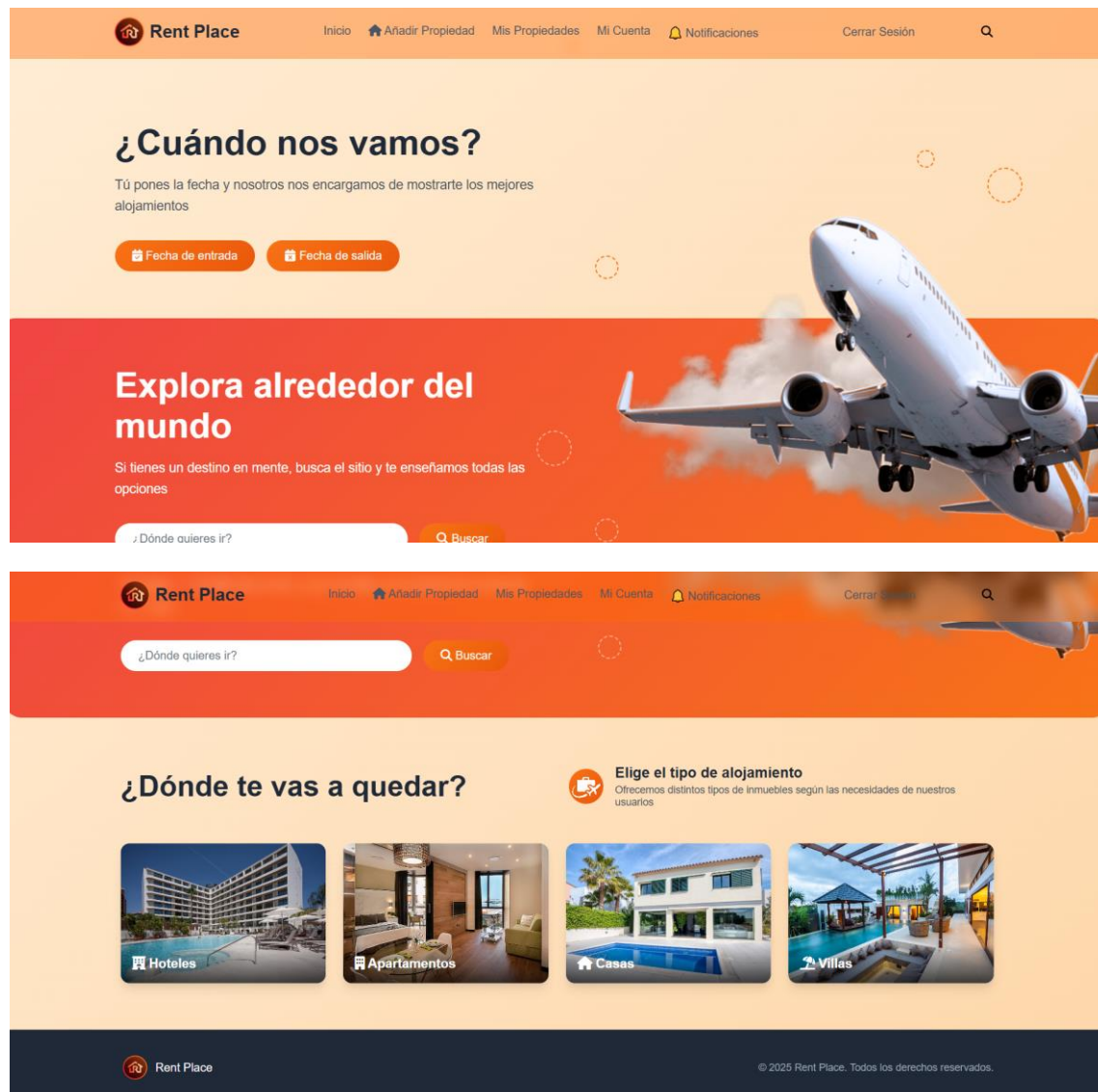


DIAGRAMA DE SECUENCIA



PROTOTIPO DEL SITIO WEB

PÁGINA PRINCIPAL



INICIO DE SESIÓN Y CREAR CUENTA

Inicio de sesión

Nombre de usuario

Contraseña

Entrar

¿No tienes cuenta? [Regístrate aquí](#)

Crear cuenta

Nombre de usuario

Contraseña

Confirmar contraseña

Teléfono

Correo electrónico

Rol

Selecciona tu rol

Registrarse

¿Ya tienes cuenta? [Inicia sesión](#)

BÚSQUEDA DE VIVIENDA

Rent Place

[Inicio](#) [Añadir Propiedad](#) [Mis Propiedades](#) [Mi Cuenta](#) [Notificaciones](#) [Cerrar Sesión](#)

Encuentra tu Alojamiento Perfecto

Busca y filtra entre miles de propiedades disponibles en todo el mundo

Fecha de entrada: dd/mm/aaaa

Fecha de salida: dd/mm/aaaa

Tipo de alojamiento: Todos los tipos

Ciudad: Ej: Madrid, Barcelona...

Buscar alojamientos **Limpiar filtros**

Resultados (4)

Three placeholder cards for search results, each with a house icon.

VISTA INQUILINO

**Mi Cuenta**

Bienvenido, a
Desde aquí puedes gestionar tu cuenta y tus propiedades.
Eres inquilino

**Mis Propiedades**
Administra las propiedades que tienes en Rentplace
[Ver mis propiedades](#)

**Añadir Propiedad**
Publica una nueva vivienda en Rentplace
[Añadir propiedad](#)

**Lista de Deseos**
Consulta tus propiedades favoritas
[Ver lista de deseos](#)

**Mis Reservas**
Consulta y gestiona tus reservas activas
[Ver mis reservas](#)

**Cambiar Rol**
Convertirte en propietario

**Rent Place**

[Inicio](#) [Añadir Propiedad](#) [Mis Propiedades](#) [Mi Cuenta](#) [Notificaciones](#) [Cerrar Sesión](#) 

**Donpablo**
Roque
€20,00 por noche
[Ver detalles](#)



**Casasaas**
Alli
€77,00 por noche
[Ver detalles](#)

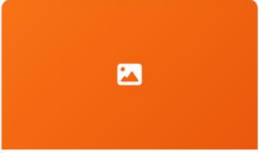
**Gestor**
Toledo
€67,00 por noche
[Ver detalles](#)

**Casita**
Pueblo
€45,00 por noche
[Ver detalles](#)

**Rent Place**

[Inicio](#) [Añadir Propiedad](#) [Mis Propiedades](#) [Mi Cuenta](#) [Notificaciones](#) [Cerrar Sesión](#) 

Resultados (4)



**Donpablo**
Roque
€20,00 por noche
[Ver detalles](#) [Lista](#)

**Gestor**
Toledo
€67,00 por noche
[Ver detalles](#) [Lista](#)

**Casita**
Pueblo
€45,00 por noche
[Ver detalles](#) [Lista](#)



14

♥ Mi Lista de Deseos

casa

Donpablo

📍

Roque

🛏

3 habitación(es)

👤

4 personas

💰

€20.00 por noche

Ver detalles

Eliminar

apartamento

Gestor

📍

Toledo

🛏

4 habitación(es)

👤

4 personas

💰

€67.00 por noche

Ver detalles

Eliminar

+ Añadir más a la lista

© 2025 Rentplace - Alquiler de viviendas

📄 Mis Reservas

Reserva #1
Alojamiento ID: 1
Desde: 20/12/2025
Hasta: 22/12/2025
Estado: Pendiente de confirmación
Total: €40.00

© 2025 Rentplace - Alquiler de viviendas

VISTA PROPIETARIO

+ Añadir Nueva Propiedad

Información Básica

Título *

Un título descriptivo para tu propiedad (máx. 150 caracteres)

Descripción

Describe las características de tu propiedad (máx. 500 caracteres)

Foto de la propiedad

Seleccionar archivo

 Ningún archivo seleccionado

Sube una imagen representativa de tu propiedad (opcional)

Ubicación

Dirección *

Características

Tipo de inmueble *

Selecciona un tipo

Habitaciones *

Número de habitaciones

Capacidad *

Número máximo de personas

Precios y Políticas

Precio por noche (€) *

Precio en euros por noche

Política de cancelación *

Selecciona una política

☐ **Permitir reserva inmediata** Los inquilinos podrán reservar directamente sin confirmación previa

☒ **Publicar Propiedad**

Mis Propiedades

[+ Añadir Nueva Propiedad](#)

Donpablo

📍 Roque

🛏 3 habitación(es) 👤 4 personas

💰 €20.00 por noche

☒ Activa [Reserva Inmediata](#)

[Ver detalles](#) [Editar](#) [Desactivar](#)

Gestor

📍 Toledo

🛏 4 habitación(es) 👤 4 personas

💰 €67.00 por noche

☒ Activa

[Ver detalles](#) [Editar](#) [Desactivar](#)

Casita

📍 Pueblo

🛏 5 habitación(es) 👤 5 personas

💰 €45.00 por noche

☒ Activa

[Ver detalles](#) [Editar](#) [Desactivar](#)

Villas

DETALLES Y RESERVA

RentPlace

[← Volver al listado](#)

Donpablo

📍 Mi casa, Roque

**20.00€** / noche

LLEGADA

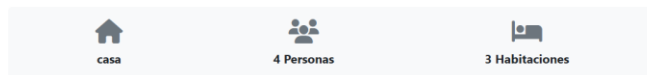
dd/mm/aaaa

SALIDA

dd/mm/aaaa

[Reservar ahora](#)[❤ Añadir a lista de deseos](#)

No se hará ningún cargo todavía.



Sobre este alojamiento

Una bonita vivienda

Política de cancelación
REEMBOLSABLE



ESTRUCTURA DE LA APLICACIÓN

-Para empezar, usaremos Spring Boot como marco de desarrollo y JPA para estructurar los datos de la base de datos Derby.

-En primer lugar, tenemos la carpeta **src/main/java** que contiene dos paquetes, el primer paquete contiene el arranque del Spring Boot y el servlet container. Luego en el segundo paquete, tiene los controladores que devuelven las vistas de Home, Login y Registrer. Por último, hay otros tres paquetes para gestionar las entidades (definición de la entidad, operaciones y capa intermedia entre los controladores y la base de datos).

-En segundo lugar, tenemos la carpeta **src/main/resources** que contiene a su vez dos carpetas más (static y templates). Dentro de la carpeta static tenemos hojas de estilo para cada página, fuentes locales y recursos estáticos como: CSS, imágenes.... Por último, en la carpeta templates contiene plantillas HTML que devuelven los controladores para mostrar las páginas al usuario.

-En tercer lugar, tenemos la carpeta **src/test/java** donde contiene el código de pruebas del proyecto para verificar que la aplicación arranca sin errores y servir como punto de partida para añadir test unitarios o de integración.

-Además, tenemos la carpeta **rentplaceDB** donde se almacenan los archivos físicos de la base de datos.

-Por último, tenemos el archivo **pom.xml** que es la configuración principal de Maven donde definimos el nombre del proyecto, sus dependencias y versión de java.

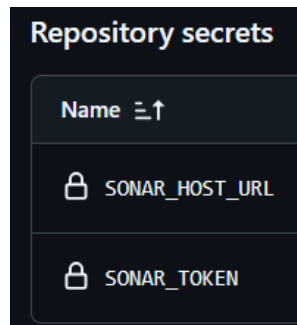
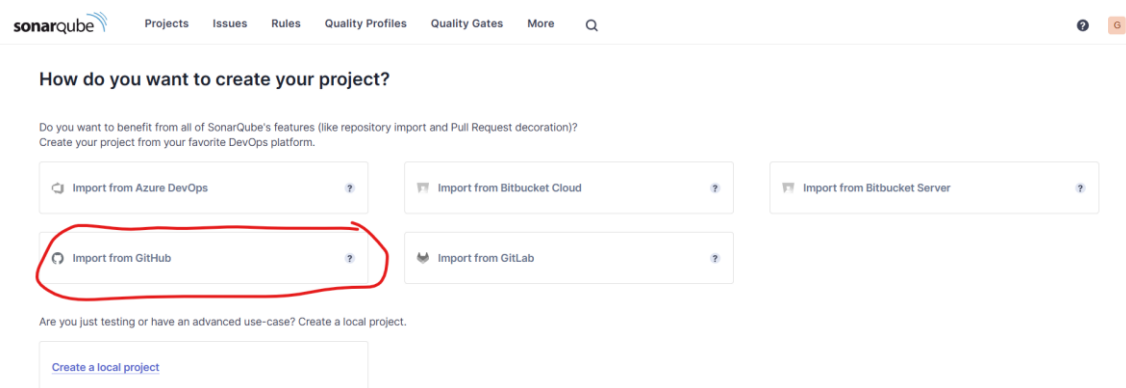
ANÁLISIS DEL SOFTWARE

SONARCLOUD

Para poder realizar los análisis de nuestro software usaremos Sonarcloud, una plataforma online que analiza automáticamente código de proyectos para detectar problemas de calidad, errores y vulnerabilidades de seguridad.

VINCULAR GITHUB CON SONARQUBE

Se vinculó sonar de manera manual con github actions, teniendo así que añadir los secrets para el token y la url de sonar:



Después de tener todo eso y haber añadido el “build.yml”, además de configurar este mismo para que analice las ramas que se pongan, hemos hecho los análisis con el siguiente comando:

```
mvn clean verify sonar:sonar \ -Dsonar.projectKey=RentPlace \ -Dsonar.projectName='RentPlace' \ -Dsonar.host.url=http://172.20.48.132:9000 \ -Dsonar.token=sqp_f5e555ac2ae6265e6e0cbfcd046045d5f66f1a1c
```

QUALITY PROFILE

Cuando hagamos un análisis, este nos proporcionará estadísticas de nuestro proyecto en base a una serie de reglas que vienen dadas por defecto. En nuestro caso, decidimos modificar ese conjunto de reglas para que se adaptará lo máximo posible a nuestros objetivos. Para ello creamos un Quality Profile, denominado como Rentplace, con java como lenguaje seleccionado.

Después de crear el Quality Profile, miramos las reglas posibles e iremos añadiendo las reglas que creamos convenientes. Para explicar las reglas las clasificamos por grupos:

➔ Seguridad:

A secure password should be used when connecting to a database
Responsibility
Security
Java - Vulnerability - cwe
Deactivate

Credentials should not be hard-coded
Responsibility
Security
Java - Vulnerability - cwe
Deactivate

Persistent entities should not be used as arguments of "@RequestMapping" methods
Consistency
Security
Java - Vulnerability - spring - cwe
Deactivate

Hemos elegido estas reglas ya que son críticas para garantizar la seguridad y robustez del código, como evitar la exposición de credenciales en texto plano y proteger con accesos no autorizados.

➔ Reliability:

"@Qualifier" should not be used on "@Bean" methods
Adaptability
Maintainability
Reliability
Java - Bug - spring
Deactivate

"Iterator.next()" methods should throw "NoSuchElementException"
Consistency
Reliability
Java - Bug - error-handling - unpredictable
Deactivate

Classes should not access their own subclasses during class initialization
Adaptability
Reliability
Java - Bug - cert
Deactivate

Null pointers should not be dereferenced
Intentionality
Reliability
Java - Bug - cwe - symbolic-execution - ...
Deactivate

Con estas reglas evitamos errores críticos en el tiempo de ejecución y garantizan estabilidad impidiendo malas prácticas en inyección de dependencias, manejo incorrecto de excepciones en iteradores, inicialización insegura de clases y accesos nulos.

➔ Mantenibilidad:

"@Qualifier" should not be used on "@Bean" methods
Adaptability
Maintainability
Reliability
Java - Bug - spring
Deactivate

"NullPointerException" should not be caught
Consistency
Maintainability
Java - Code Smell - error-handling - cwe - ...
Deactivate

"NullPointerException" should not be explicitly thrown
Consistency
Maintainability
Java - Code Smell - error-handling - pitfall
Deactivate

"public static" fields should be constant
Adaptability
Maintainability
Java - Code Smell - cwe - cert
Deactivate

"readResolve" methods should be inheritable
Intentionality
Maintainability
Java - Code Smell - pitfall
Deactivate

["switch case" clauses should not have too many lines of code](#)

Adaptability

Maintainability ⬇

Java • Code Smell • brain-overload

Deactivate

["switch" statements and expressions should not be nested](#)

Adaptability

Maintainability ⬇

Java • Code Smell • pitfall

Deactivate

["switch" statements should not have too many "case" clauses](#)

Adaptability

Maintainability ⬇

Java • Code Smell • brain-overload

Deactivate

["URL.hashCode" and "URL.equals" should be avoided](#)

Consistency

Maintainability ⬇

Java • Code Smell • performance

Deactivate

[A "while" loop should be used instead of a "for" loop](#)

Intentionality

Maintainability ⬆

Java • Code Smell • clumsy

Deactivate

[A conditionally executed single line should be denoted by indentation](#)

Consistency

Maintainability ⬇

Java • Code Smell • suspicious • confusing

Deactivate

[A field should not duplicate the name of its containing class](#)

Consistency

Maintainability ⬇

Java • Code Smell • brain-overload

Deactivate

[Abstract class names should comply with a naming convention](#)

Consistency

Maintainability ⬆

Java • Code Smell • convention

Deactivate

[An abstract class should have both abstract and concrete methods](#)

Consistency

Maintainability ⬆

Java • Code Smell • convention

Deactivate

[Array designators "\[\]" should be located after the type in method signatures](#)

Consistency

Maintainability ⬆

Java • Code Smell • convention

Deactivate

[Array designators "\[\]" should be on the type, not the variable](#)

Consistency

Maintainability ⬆

Java • Code Smell • convention

Deactivate

[Bean names should adhere to the naming conventions](#)

Consistency

Maintainability ⬇

Java • Code Smell • spring

Deactivate

[Boolean checks should not be inverted](#)

Consistency

Maintainability ⬆

Java • Code Smell • pitfall

Deactivate

[Boolean literals should not be redundant](#)

Consistency

Maintainability ⬆

Java • Code Smell • clumsy

Deactivate

[Catches should be combined](#)

Adaptability

Maintainability ⬆

Java • Code Smell • clumsy

Deactivate

Child class fields should not shadow parent class fields	Intentionality
Maintainability	Java • Code Smell • confusing Deactivate
Class members annotated with "@VisibleForTesting" should not be accessed from production code	Consistency
Maintainability	Java • Code Smell • pitfall Deactivate
Class names should comply with a naming convention	Consistency
Maintainability	Java • Code Smell • convention Deactivate
Class variable fields should not have public accessibility	Adaptability
Maintainability	Java • Code Smell • cwe Deactivate
Classes named like "Exception" should extend "Exception" or a subclass	Consistency
Maintainability	Java • Code Smell • error-handling convention ... Deactivate
Classes should not be coupled to too many other classes	Adaptability
Maintainability	Java • Code Smell • brain-overload Deactivate
Classes should not depend on an excessive number of classes (aka Monster Class)	Adaptability
Maintainability	Java • Code Smell • BETA design architecture Deactivate
Classes should not have too many "static" imports	Adaptability
Maintainability	Java • Code Smell • brain-overload Deactivate
Classes should not have too many fields	Adaptability
Maintainability	Java • Code Smell • brain-overload Deactivate
Classes should not have too many methods	Adaptability
Maintainability	Java • Code Smell • brain-overload Deactivate
Classes without "public" constructors should be "final"	Consistency
Maintainability	Java • Code Smell • design Deactivate
Close curly brace and the next "else", "catch" and "finally" keywords should be located on the same line	Consistency
Maintainability	Java • Code Smell • convention Deactivate
Cognitive Complexity of methods should not be too high	Adaptability
Maintainability	Java • Code Smell • brain-overload Deactivate
Conditionals should start on new lines	Consistency
Maintainability	Java • Code Smell • suspicious Deactivate
Constant names should comply with a naming convention	Consistency
Maintainability	Java • Code Smell • convention Deactivate

Constructors should not be used to instantiate "String", "BigInteger", "BigDecimal" and primitive-wrapper classes

Consistency

Maintainability

Java • Code Smell • performance

Deactivate

Control flow statements "if", "for", "while", "switch" and "try" should not be nested too deeply

Adaptability

Maintainability

Java • Code Smell • brain-overload

Deactivate

Declarations should use Java collection interfaces such as "List" rather than specific implementation classes such as "LinkedList"

Consistency

Maintainability

Java • Code Smell • bad-practice

Deactivate

Enum values should be compared with "=="

Consistency

Maintainability

Java • Code Smell

Deactivate

Exception classes should have final fields

Consistency

Maintainability

Java • Code Smell • error-handling

Deactivate

Exception handlers should preserve the original exceptions

Consistency

Maintainability

Java • Code Smell • cwe error-handling ...

Deactivate

Exception types should not be tested using "instanceof" in catch blocks

Consistency

Maintainability

Java • Code Smell • error-handling cert ...

Deactivate

Exceptions should be either logged or rethrown but not both

Adaptability

Maintainability

Java • Code Smell • error-handling clumsy

Deactivate

Expressions should not be too complex

Adaptability

Maintainability

Java • Code Smell • brain-overload

Deactivate

Extensions and implementations should not be redundant

Intentionality

Maintainability

Java • Code Smell • clumsy

Deactivate

Field names should comply with a naming convention

Consistency

Maintainability 

Java •  Code Smell • convention

Deactivate

Future keywords should not be used as names

Consistency

Maintainability 

Java •  Code Smell • obsolete pitfall

Deactivate

Inheritance tree of classes should not be too deep

Adaptability

Maintainability 

Java •  Code Smell • design

Deactivate

Inner classes should not have too many lines of code

Adaptability

Maintainability 

Java •  Code Smell • brain-overload

Deactivate

Interfaces should not solely consist of constants

Adaptability

Maintainability 

Java •  Code Smell • bad-practice

Deactivate

JUnit test cases should call super methods

Intentionality

Maintainability 

Java •  Code Smell • junit tests

Deactivate

Labels should not be used

Consistency

Maintainability 

Java •  Code Smell • confusing

Deactivate

Lambdas containing only one statement should not nest this statement in a block

Intentionality

Maintainability 

Java •  Code Smell • java8

Deactivate

Lines should not be too long

Consistency

Maintainability 

Java •  Code Smell • convention

Deactivate

Literal suffixes should be upper case

Consistency

Maintainability 

Java •  Code Smell • convention cert ...

Deactivate

Local constants should follow naming conventions for constants

Consistency

Maintainability 

Java •  Code Smell • convention

Deactivate

Local variable and method parameter names should comply with a naming convention

Consistency

Maintainability 

Java •  Code Smell • convention

Deactivate

Loggers should be "private static final" and should share a naming convention

Consistency

Maintainability 

Java •  Code Smell • convention

Deactivate

Loops should not contain more than a single "break" or "continue" statement

Adaptability

Maintainability 

Java •  Code Smell • brain-overload

Deactivate

Methods and field names should not be the same or differ only by capitalization

Consistency

Maintainability 

Java •  Code Smell • confusing

Deactivate

[Methods should not be too complex](#)

Adaptability

Maintainability 🚫

Java • ☞ Code Smell • brain-overload

Deactivate

[Methods should not have identical implementations](#)

Adaptability

Maintainability 🚫

Java • ☞ Code Smell • confusing duplicate ...

Deactivate

[Methods should not have too many lines](#)

Adaptability

Maintainability 🚫

Java • ☞ Code Smell • brain-overload

Deactivate

[Methods should not have too many parameters](#)

Adaptability

Maintainability 🚫

Java • ☞ Code Smell • brain-overload

Deactivate

[Methods should not perform too many tasks \(aka Brain method\)](#)

Adaptability

Maintainability 🟢

Java • ☞ Code Smell • BETA • design architecture

Deactivate

[Multiline blocks should be enclosed in curly braces](#)

Consistency

Maintainability 🚫

Java • ☞ Code Smell • cwe cert

Deactivate

[Multiple variables should not be declared on the same line](#)

Consistency

Maintainability 🟢

Java • ☞ Code Smell • convention cert

Deactivate

[Mutable fields should not be "public static"](#)

Consistency

Maintainability 🟢

Java • ☞ Code Smell • cwe unpredictable ...

Deactivate

[Package declaration should match source file directory](#)

Consistency

Maintainability 🚫

Java • ☞ Code Smell • pitfall

Deactivate

[Package names should comply with a naming convention](#)

Consistency

Maintainability 🟢

Java • ☞ Code Smell • convention

Deactivate

[Parsing should be used to convert "Strings" to primitives](#)

Consistency

Maintainability 🟢

Java • ☞ Code Smell • performance

Deactivate

[Public methods should not contain selector arguments](#)

Adaptability

Maintainability 🚫

Java • ☞ Code Smell • design

Deactivate

[Public methods should throw at most one checked exception](#)

Adaptability

Maintainability 🚫

Java • ☞ Code Smell • error-handling

Deactivate

[Similar tests should be grouped in a single Parameterized test](#)

Consistency

Maintainability 🚫

Java • ☞ Code Smell • tests bad-practice ...

Deactivate

[Simple class names should be used](#)

Consistency

Maintainability 🟢

Java • ☞ Code Smell • confusing redundant

Deactivate

[Single-character alternations in regular expressions should be replaced with character classes](#)

Intentionality

Maintainability ⬇

Java • ☹ Code Smell • regex

Deactivate

[Source code should be indented consistently](#)

Consistency

Maintainability ⬇

Java • ☹ Code Smell • convention

Deactivate

[Statements should be on separate lines](#)

Consistency

Maintainability ⬇

Java • ☹ Code Smell • convention

Deactivate

[Static non-final field names should comply with a naming convention](#)

Consistency

Maintainability ⬇

Java • ☹ Code Smell • convention

Deactivate

[String literals should not be duplicated](#)

Adaptability

Maintainability ⬇

Java • ☹ Code Smell • design

Deactivate

[String multiline concatenation should be replaced with Text Blocks](#)

Consistency

Maintainability ⬇

Java • ☹ Code Smell • java15

Deactivate

[Synchronized classes "Vector", "Hashtable", "Stack" and "StringBuffer" should not be used](#)

Consistency

Maintainability ⬇

Java • ☹ Code Smell • performance

Deactivate

[Test classes should comply with a naming convention](#)

Consistency

Maintainability ⬇

Java • ☹ Code Smell • convention tests

Deactivate

[Test methods should comply with a naming convention](#)

Consistency

Maintainability ⬇

Java • ☹ Code Smell • convention tests

Deactivate

[Test methods should not contain too many assertions](#)

Adaptability

Maintainability ⬇

Java • ☹ Code Smell • tests

Deactivate

[TestCases should contain tests](#)

Adaptability

Maintainability ⬇

Java • ☹ Code Smell • junit tests ...

Deactivate

[Tests should be kept in a dedicated source directory](#)

Adaptability

Maintainability ⬇

Java • ☹ Code Smell • tests suspicious

Deactivate

[Tests should be stable](#)

Adaptability

Maintainability ⬇ Reliability ⬇

Java • ☹ Code Smell • tests unpredictable ...

Deactivate

[Tests should include assertions](#)

Adaptability

Maintainability ⬇

Java • ☹ Code Smell • junit tests

Deactivate

[Text blocks should not be used in complex expressions](#)

Adaptability

Maintainability ⬇

Java • ☹ Code Smell

Deactivate

The default unnamed package should not be used	Adaptability
Maintainability	Java • Code Smell • convention
	Deactivate
The members of an interface or class declaration should appear in a pre-defined order	Consistency
Maintainability	Java • Code Smell • convention
	Deactivate
The names of methods with boolean return values should start with "is" or "has"	Consistency
Maintainability	Java • Code Smell • convention
	Deactivate
The ternary operator should not be used	Consistency
Maintainability	Java • Code Smell • brain-overload
	Deactivate
Threads should not be started in constructors	Adaptability
Maintainability	Java • Code Smell • multi-threading cert ...
	Deactivate
Throwable and Error should not be caught	Consistency
Maintainability	Java • Code Smell • cwe error-handling ...
	Deactivate
Try-with-resources should be used	Consistency
Maintainability	Java • Code Smell • java8 cert ...
	Deactivate
Two branches in a conditional structure should not have exactly the same implementation	Adaptability
Maintainability	Java • Code Smell • design suspicious
	Deactivate
Type parameter names should comply with a naming convention	Consistency
Maintainability	Java • Code Smell • convention
	Deactivate
Unnecessary semicolons should be omitted	Consistency
Maintainability	Java • Code Smell • convention
	Deactivate

Creemos que este es el conjunto de reglas de mantenibilidad más adecuado y que nos ayuda a evitar falsos positivos. Todas estas reglas nos permiten eliminar los codes smells y las bad-practices, pudiendo reducir así la deuda técnica.

QUALITY GATE

Hemos configurado las condiciones **on New Code** en el Quality Gate para garantizar estándares realistas de calidad y seguridad, adaptados a nuestro equipo con menos experiencia, mediante estas condiciones:

- ➔ **Issues > 0** evitamos la introducción de errores críticos, manteniendo un umbral mínimo para no sobrecargar al equipo.
- ➔ **Security Hotspots Reviewed < 100%** aseguramos que al menos el 90% de riesgos sean revisados, priorizando seguridad sin exigir perfección.
- ➔ **Coverage < 70.0%** garantizamos cobertura razonable en pruebas, equilibrando calidad y velocidad de desarrollo.
- ➔ **Duplicated Lines (%) > 5.0%** limitamos el código duplicado para mejorar mantenibilidad sin penalizar excesivamente.

- ➔ **Bugs > 3 y Vulnerabilities > 1** controlamos errores y riesgos críticos, permitiendo hasta 3 bugs y 1 vulnerabilidad para equipos en formación.
- ➔ **Reliability/Security Rating is worse than B** alineamos con prácticas alcanzables, evitando estándares inalcanzables para nuestro equipo con poca experiencia.

Conditions on New Code

Metric	Operator	Value
Issues	is greater than	0
Security Hotspots Reviewed	is less than	100%
Coverage	is less than	70.0%
Duplicated Lines (%)	is greater than	5.0%
Bugs	is greater than	3
Vulnerabilities	is greater than	1
Reliability Rating	is worse than	B
Security Rating	is worse than	B

En conclusión, prevenimos de forma temprana los bugs y vulnerabilidades en el flujo de integración continua, reducimos la deuda técnica al bloquear merges con código de baja calidad. Y, por último, cumplimos de forma automática los estándares de seguridad y fiabilidad.

Hemos configurado las condiciones **on Overall Code** en el Quality Gate para garantizar estándares realistas de calidad y seguridad, adaptados a nuestro equipo con menos experiencia, mediante estas condiciones:

- ➔ **Blocker Issues > 2** permite hasta 2 errores críticos, evitando bloqueos excesivos en código legacy.
- ➔ **Condition Coverage < 40.0%** exige una cobertura mínima en ramas lógicas, priorizamos estabilidad sin sobrecargar al equipo.
- ➔ **Critical Issues > 5** tolera hasta 5 vulnerabilidades críticas, enfocándonos en resolver las más urgentes primero.
- ➔ **Duplicated Lines (%) > 10.0%** acepta duplicación controlada en código existente, priorizando refactorización progresiva.
- ➔ **Line Coverage < 50.0%** establecemos un mínimo razonable de cobertura para pruebas, compatible con la curva de aprendizaje del equipo.
- ➔ **Maintainability/Reliability/Security Rating worse than C** permite calificaciones funcionales C, y así evitamos exigir estándares inalcanzables para nuestro equipo con poca experiencia.
- ➔ **Major Issues > 15** limita errores menores a 15, equilibramos calidad y velocidad de entrega en proyectos con deuda técnica acumulada.

Conditions on Overall Code

Metric	Operator	Value
Blocker Issues	is greater than	2
Condition Coverage	is less than	40.0%
Critical Issues	is greater than	5
Duplicated Lines (%)	is greater than	10.0%
Line Coverage	is less than	50.0%
Maintainability Rating	is worse than	C
Major Issues	is greater than	15
Reliability Rating	is worse than	C
Security Rating	is worse than	C

TESTING

Para realizar el testing hemos integrado Junit, Surefire y Jacoco.

En primer lugar, Junit es un marco utilizado para escribir y ejecutar pruebas automatizadas de pequeñas unidades de código, como métodos o clases. Por lo que, nos permite facilitar la verificación de nuestro código creando pruebas que comprueban la lógica y ayudan a identificar errores de manera temprana.

Por otro lado, SureFire es un plugin de Maven que se utiliza para ejecutar pruebas unitarias en proyectos de Java. Su función principal es ejecutar las pruebas definidas durante la fase de test. En nuestro proyecto será utilizado como entorno de pruebas.

Por último, JaCoCo es una librería de Java creada para la evaluación de cobertura de sentencia. Esta se puede integrar con varias librerías de pruebas. Por lo que podemos implementarla en nuestro proyecto, dado que es compatible con JUnit.

Para integrar las herramientas de reporting y análisis de calidad en nuestro proyecto, se han configurado los plugins necesarios en el pom.xml. Se incluye maven-surefire-report-plugin (versión **3.2.2**) para generar reportes detallados de las pruebas unitarias, junto con maven-site-plugin, que estructura un sitio web HTML con la documentación del proyecto. Para habilitar referencias cruzadas al código fuente (navegación entre clases y líneas específicas), se integra maven-jxr-plugin en la configuración del sitio. La cobertura de código se gestiona mediante jacoco-maven-plugin (versión **0.8.12**), configurado tanto en la fase de construcción (<build>) para instrumentar las pruebas, como en la sección <reporting> para incluir sus métricas en el sitio generado.

Las versiones de los plugins se centralizan en <properties> con esto garantizamos consistencia, y toda la configuración se organiza para que, al ejecutar mvn site, se generen automáticamente los reportes de pruebas, cobertura y documentación cruzada en target/site. Esta estructura asegura una integración limpia con Spring Boot 3.5.6 y compatibilidad con Java 21.

GESTOR PROPIEDAD

Enlace: https://docs.google.com/spreadsheets/d/1rKcG5PK--9P0cCzxFX4_3qpf0zj7aNR8/edit?usp=sharing&ouid=112655821874894459423&rtpof=true&sd=true

GESTOR RESERVA

Enlace: <https://docs.google.com/spreadsheets/d/1xJ4Hm6rs98nvGVCNugoZDTd7v-J2jHFI/edit?usp=sharing&ouid=112655821874894459423&rtpof=true&sd=true>

GESTOR LISTA DESEOS

Enlace: <https://docs.google.com/spreadsheets/d/1cLPGtgUQg77XeJP4FIRXOuS8y8vMkdKZ/edit?usp=sharing&ouid=112655821874894459423&rtpof=true&sd=true>

GESTOR BÚSQUEDA

Enlace: https://docs.google.com/spreadsheets/d/1VUOaPk8RUc1PbbfZxi_NslGhb3PCFKAT/edit?usp=sharing&ouid=109367345431997350888&rtpof=true&sd=true

GESTOR ROL

Enlace: https://docs.google.com/spreadsheets/d/e/2PACX-1vTJ83hkREgKw8JqVw5zClbVkyUpsRRxdsCXzpWe-FzmDIVC_7dPJ2-6h1RfPoipzw/pub?output=xlsx

GESTOR USUARIO

Enlace: https://docs.google.com/spreadsheets/d/1VUOaPk8RUc1PbbfZxi_NslGhb3PCFKAT/edit?usp=sharing&ouid=109367345431997350888&rtpof=true&sd=true

GESTOR NOTIFICACIÓN

Enlace: https://docs.google.com/spreadsheets/d/1E9Y2jtQsfJWLTUQ_18iyy-kt8C3vxNHD/edit?usp=sharing&ouid=112655821874894459423&rtpof=true&sd=true

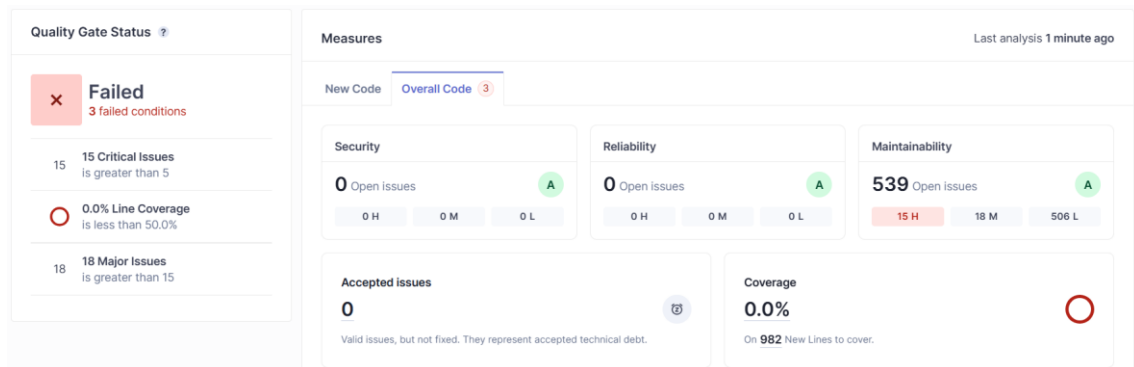
GESTOR DISPONIBILIDAD

Enlace: <https://docs.google.com/spreadsheets/d/1ZMRw2g2s3IHM6gJWHgVUYHIUjxm9vV-n/edit?usp=sharing&ouid=112655821874894459423&rtpof=true&sd=true>

rentplace

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
es.ucdm.rentplace.controller		0 %		0 %	138	138	435	435	50	50	10	10
es.ucdm.rentplace.service		0 %		0 %	130	130	240	240	78	78	9	9
es.ucdm.rentplace.entity		11 %		0 %	148	173	248	289	141	166	7	11
es.ucdm.rentplace.gestorestest		87 %		44 %	116	227	58	709	93	201	0	9
es.ucdm.rentplace.gestores		91 %		78 %	93	331	38	428	34	191	3	32
es.ucdm.rentplace.dto		0 %		0 %	17	17	22	22	13	13	1	1
es.ucdm.rentplace		0 %		n/a	5	5	6	6	5	5	2	2
Total	4.490 of 10.012	55 %	390 of 634	38 %	647	1.021	1.047	2.129	414	704	32	74

Generamos el informe jacoco con el comando **“mvn clean test jacoco:report -Djacoco.append=false”** y ejecutamos el index.html con **“start target\site\jacoco\index.html”** y nos sale total un 55%, ya que controller y service tiene 0% cobertura porque no tienen test para código de producción.



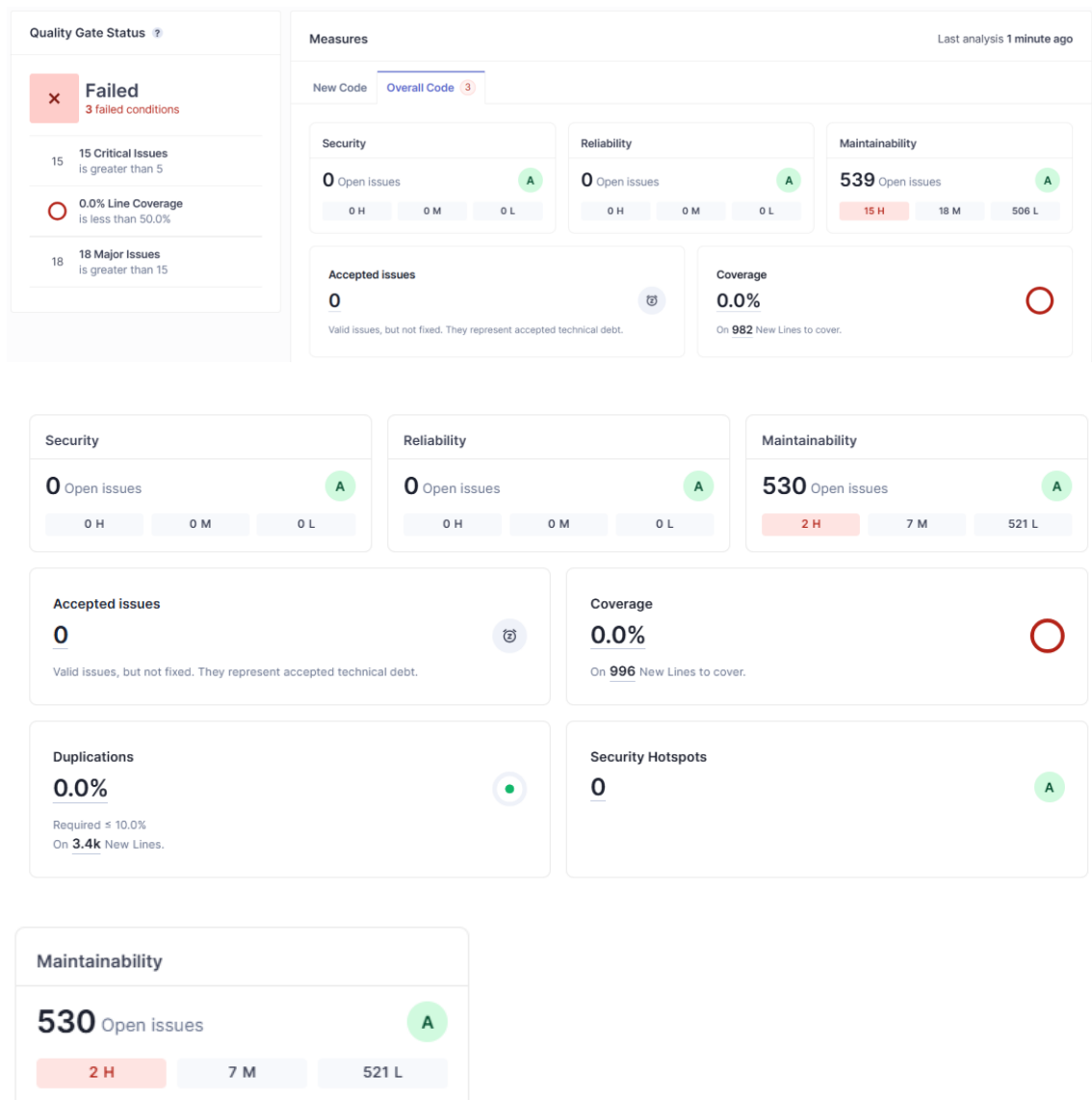
Pero no hemos conseguido que se refleje en sonar.

MANTENIMIENTO

En primer lugar, elegimos un tipo de plan de mantenimiento que más nos encaje a nuestro proyecto. Por lo que elegimos el **Mantenimiento Preventivo**. Con este nos permite prevenir problemas a futuros mediante la identificación y resolución de posibles puntos débiles en nuestro software antes de que se conviertan en fallos. En el primer análisis de prueba nos salía Critical Issues y Major Issues, además de 0% de cobertura ya que no habíamos hechos testing. Nuestro objetivo ahora es reducir lo máximo posibles los issues y aumentar la cobertura lo máximo posible.

➔ **Issues corregidas:**

- Reemplazado el operador ternario (? :) por estructuras if/else explícitas.
- Unidas las dos condiciones idénticas en una sola expresión con operador ||.
- Eliminada la duplicación de bloques if/else if que asignaban el mismo valor (true).
- Se separó la sentencia if (reserva == null) return BigDecimal.ZERO; en un bloque if multilinea para cumplir con la regla de una sentencia por línea.
- Se reemplazaron las repeticiones de literales por constantes estáticas para evitar duplicación y facilitar el mantenimiento.



De tener **15 issues de Alto Impacto** y **18 Issues de Impacto Medio**.

Hemo reducido a **2 issues de Alto Impacto** y a **7 issues de Impacto Medio**.